

Кооперативная отмена асинхронных задач

Механизм отмены называется кооперативным, поскольку для его работы нужны изменения как в вызывающем, так и в вызываемом коде. Вызывающий код создает источник токенов отмены и передает токен в вызываемый код. Вызываемый код в "узловых" точках проверяет, было ли запрошено прерывание, и выполняет "мягкое" и безопасное прерывание, если необходимо.

Для минимизации изменений в прикладном коде необходимые проверки включены в вызовы RPC методов, обращения к БД и т.д.

Основой фреймворка являются классы [StopToken](#), [StopSource](#) и `ExecutionInterruptedException`.

Класс `StopToken` предоставляет различные методы ожидания прерывания:

- `StopToken::WaitForDataOrInterruption()` - возвращает результат асинхронной операции, если она завершилась, либо генерирует исключение, которое сигнализирует, что произошла отмена операции (используется при ожидании одной операции).
- `StopToken::WaitForAnyDataOrInterruption()` - аналогичен методу `WaitForDataOrInterruption()` (используется при ожидании нескольких операций).
- `StopToken::WaitForInterruption()` - ожидает отмены операции.
- `StopToken::InterruptionRequested()` - возвращает значение флага прерывания.
- `StopToken::CanBeInterrupted()` - проверяет, возможно ли прерывание данного токена.
- `StopToken::InterruptionPoint()` - проверяет, запрошено ли прерывание и выбрасывает исключение `ExecutionInterruptedException`, если прерывание запрошено.

Исключение `ExecutionInterruptedException` выбрасывается в точках прерывания. Данное исключение является предупреждением и не генерирует ошибок в логах сервиса.

Класс `StopSource` предназначен для создания прерываемых `StopToken`'ов и инициации прерывания.

- `StopSource::GetToken()` - позволяет получить токен отмены.
- `StopSource::RequestInterruption()` - устанавливает флаг прерывания. При вызове данного метода все токены, полученные с помощью `GetToken()`, получают сигнал о прерывании.
- `StopSource::InterruptionRequested()` - возвращает значение флага прерывания.

Класс `StopSource` также предоставляет статический метод `StopSource::CreateLinkedStopSource()`, создающий связанный `StopSource`, который будет прерван при прерывании любого из переданных токенов.



Флаг прерывания, возведенный у `StopSource`, уже нельзя сбросить. Это можно сделать только при создании нового `StopSource`.

Вспомогательный класс [StopCallback](#) позволяет установить коллбек на срабатывание токена отмены.

Источники токенов прерывания по умолчанию

Для каждого вызова RPC метода платформа генерирует токен отмены "по умолчанию". Этот токен можно получить при помощи функции [DefaultContextStopToken](#). Токен отмены по умолчанию формируется как комбинация токенов, которые возвращаются источниками прерываний, зарегистрированными в сервисе.

По умолчанию в платформе реализованы следующие источники прерываний:

1. Источник, реагирующий на разрыв соединения с клиентом.
2. Источник, реагирующий на остановку сервиса.
3. Источник, реагирующий на отмену сценария DWC, в рамках которого вызван метод.

Реализация источников токенов прерывания

```
1 class ContextStopTokenProvider
```

```

2 {
3 public:
4     virtual std::unique_ptr< ContextStopTokenHolder >
        GetStopTokenForContext( StringView method, size_t num_args, IContext
        const& ctx, HttpHeaders const& h ) const = 0;
5 };

```

Вызов `GetStopTokenForContext()` возвращает указатель на объект [ContextStopTokenHolder](#), который предоставляет метод `GetToken()` для получения токена отмены и может реализовывать логику очистки при разрушении токена.

Компоненты платформы поддерживающие токены отмены

Методы `Invoke` в `blcore::Object`

Все версии метода `Invoke` поддерживают передачу токена отмены в качестве параметра. В коде на C++ единственное ограничение - токен отмены не должен быть первым параметром, в коде на Python токен отмены передается в именованном параметре `stop_token`.

Если токен отмены не передан в явном виде, то будет использован токен отмены "[по умолчанию](#)", возвращаемый функцией `DefaultContextStopToken`. Токены отмены могут передаваться как в локальные, так и в удаленные вызовы.

Пример C++:

```

1 StopToken st;
2 blcore::Object obj( L"Object" );
3 obj.Invoke< void >( L"Method", st );

```

Пример Python:

```

1 stop_token = sbis.StopToken()
2 BLObject("Object").Invoke("Method", stop_token=stop_token)

```

Таймаут для локального вызова

С реализаций кооперативной отмены появилась возможность устанавливать таймауты для локальных вызовов RPC методов. Таймаут устанавливается стандартным методом `SetTimeout()`. Так как таймаут реализован через кооперативную отмену, завершение метода точно по истечении указанного времени не гарантируется.

Клиент HTTP

Как синхронная, так и асинхронная версии платформенного HTTP клиента, поддерживают токены отмены. В асинхронном клиенте токен отмены задается методом `async_http::Request::SetStopToken()` для запроса, в синхронном клиенте существуют перегрузки методов, принимающие токен отмены. HTTP-клиент не использует токен отмены по умолчанию, т.е. токен отмены всегда должен задаваться явно. В случае, если запрос был прерван, HTTP клиент возвращает статус `HttpStatus::INTERRUPTED_BY_STOP_TOKEN`.

Пример C++:

```

1 StopSource ss;
2 // асинхронный клиент
3 auto request = std::make_unique< communication::async_http::Request >(
    L"http://sbis.ru/");
4 request->SetStopToken( ss.GetToken() );
5 auto cln = std::make_unique< communication::async_http::Client >();
6 cln->Get( std::move( request ) );
7
8 // синхронный клиент
9 auto client = CreateHTTPClientUnique();
10 client->Get( L"http://sbis.ru/", ss.GetToken() );

```

Пример Python:

```
1 # асинхронный клиент
2 ss = sbis.StopSource()
3 r = sbis.AsyncHTTPRequest( "http://sbis.ru" )
4 r.SetStopToken(ss.GetToken())
5 cln = sbis.AsyncHTTPClient()
6 cln.Get(r)
7
8 # синхронный клиент
9 cln = sbis.IHTTPClient()
10 result = cln.Get( "http://sbis.ru/", ss.GetToken() )
```

Работа с БД

Поддержка кооперативной отмены внедрена в драйверы БД PostgreSQL, SQLite, Clickhouse. Токены отмены могут быть явно переданы в методы классов [dbi::IDatabase](#), [dbi::IStatement](#), [dbi::ITableCopier](#). Если токен отмены не передан, то используется токен по умолчанию для текущего контекста исполнения.

Пример C++:

```
1 // dbi::IStatement
2 StopSource stop_source;
3 auto statement = db->CreateStatement();
4 statement->Exec( L"SELECT pg_sleep(5)"_sv, stop_source.GetToken() );
5
6 // dbi::IDatabase
7 StopSource stop_source;
8 db->Connect( stop_source.GetToken() );
9
10 // dbi::ITableCopier
11 auto copier = db->CreateTableCopier( format );
12 StopSource stop_source;
13 for( int i = 0; i < ROW_COUNT; ++i )
14     *copier << /* data */;
15 auto future = copier->FlushAsync( stop_source.GetToken() );
16
17 // SqlQuery
18 SqlQuery( stop_token, L"select pg_sleep(5)" );
```

Пример Python:

```
1 # SqlQuery
2 stop_source = StopSource()
3 SqlQuery(stop_source.GetToken(), 'SELECT pg_sleep(5)')
4
5 # AsyncQuery
6 stop_source = StopSource()
7 stmt = CreateStatement()
8 with stmt.AsyncQuery('select pg_sleep(5)', stop_source.GetToken()) as
   result:
9     cursor = result.Next()
```

Клиент Redis

В синхронном клиенте Redis реализованы перегрузки методов, принимающих токен отмены. В асинхронном клиенте токен отмены передается в методы `redis::AsyncClient::SendCommand` и `redis::AsyncClient::SendCommandsAtomic`.

Пример C++:

```

1 // Синхронный клиент
2 redis::Client client( L"RedisCluster", true );
3 StopSource stop_source;
4 client.SetValue( key, value, stop_source.GetToken());
5 client.IsExists( key, stop_source.GetToken());
6
7 // Асинхронный клиент
8 redis::AsyncClient client{ AsyncClient::FromClusterName,
9 L"RedisCluster"_s };
9 StopSource stop_source;
10 auto bl_pop = client.SendCommand( stop_source.GetToken(),
    redis::commands::BLPop{ key, 3 } );

```

Пример Python:

```

1 stop_source = sbis.StopSource()
2 with sbis.RedisClient("RedisCluster", True, False) as client:
3     client.BLPop(key, 3, stop_source.GetToken())

```

Клиент Kafka

В виду слабой распространенности клиента Kafka в прикладной кодовой базе, кооперативная отмена реализована только в интерфейсе `IKafkaConsumer`. Если токен не передан, `IKafkaConsumer` использует токен по умолчанию для текущего контекста.

Пример C++:

```

1 StopSource stop_source;
2 auto consumer = kafka::ConsumerPool::Instance().GetOrCreate(
3 L"KafkaCluster", L"ConsumerGroup" );
3 consumer->Subscribe( { "topic" } );
4 auto record_fut = consumer->ConsumerAsync( stop_source.GetToken() );
5 auto record = record_fut->Get();

```

Платформенный класс Thread

Платформенный класс [Thread](#) представляет собой аналог `std::jthread` для создания именованных потоков, поддерживающих кооперативное прерывание. Если потоковая функция принимает `StopToken`, то он передается в функцию в явном виде, иначе связанный токен отмены можно получить при помощи функции `GetCurrentThreadStopToken`. Для запроса прерывания используется метод `Thread::RequestInterruption()`.

Пример C++:

```

1 auto thread_fn = [] ( StopToken stop_token )
2 {
3     stop_token.WaitForInterruption( Seconds{ 5 } );
4 };
5
6 {
7     Thread thread{ L"Thread"_s, thread_fn };
8     thread.RequestInterruption();
9     thread.Join();
10 }
11
12 // поток можно создавать с внешним токеном отмены
13 {
14     StopSource stop_source;
15     Thread thread{ L"Thread"_s, stop_source.GetToken(), thread_fn };
16     stop_source.RequestInterruption();
17     thread.Join();

```

Платформенный класс `ConditionVariable`

Платформенный класс [ConditionVariable](#) является аналогом `std::condition_variable_any`, предоставляющий методы `Wait`, которые могут ждать активации условной переменной или запроса прерывания.

Пример C++:

```
1 StopSource stop_source;  
2 ConditionVariable cv;  
3 std::mutex m;  
4 std::unique_lock lock{ m };  
5 cv.Wait( lock, stop_source.GetToken() );
```